

脑与认知科学大作业报告

神经网络实现编解码

班级： 人工智能2402班 学号： 202431121013030 姓名： 胡弈周

摘要

本文基于 Python 语言手动实现了一个 8-3-8 结构的自编码器神经网络，用于完成 8×8 单位矩阵的编解码任务。网络由输入层、隐藏层和输出层组成，其中输入层和输出层均包含 8 个神经元，隐藏层包含 3 个神经元。程序没有使用 TensorFlow、PyTorch 等深度学习框架，而是使用 NumPy 完成前向传播、误差反向传播和梯度下降参数更新，并使用 Matplotlib 绘制训练过程中的损失函数曲线。实验结果表明，经过训练后，网络能够将 8×8 单位矩阵压缩到 8×3 的隐藏层编码表示，并较好地恢复出原始单位矩阵。

1 作业任务说明

本次作业要求实现一个 8-3-8 的自编码器神经网络结构。网络输入为 8×8 单位矩阵，每一行为一个训练样本，也就是一个 one-hot 向量。网络需要通过隐藏层完成编码，再通过输出层完成解码，使输出结果尽可能接近原始输入。

本实验程序使用 Python 编写，主要依赖 NumPy 和 Matplotlib。其中 NumPy 用于矩阵计算和手动实现梯度下降，Matplotlib 用于绘制损失函数图像。整个训练过程没有调用 TensorFlow、PyTorch 等深度学习框架，满足手动实现神经网络训练算法的要求。

2 网络结构设计

本实验采用三层全连接神经网络：

- 输入层：8 个神经元，对应 8 维 one-hot 输入向量；
- 隐藏层：3 个神经元，用于学习输入数据的压缩编码；
- 输出层：8 个神经元，用于将隐藏层编码结果解码回 8 维输出向量。

设输入矩阵为 $X \in \mathbb{R}^{8 \times 8}$ ，真实标签矩阵为 $Y \in \mathbb{R}^{8 \times 8}$ 。由于本任务是自编码任务，因此有：

$$Y = X = I_8 \tag{1}$$

其中 I_8 表示 8×8 单位矩阵。输入层到隐藏层的权重矩阵为 $W^{(1)} \in \mathbb{R}^{8 \times 3}$ ，偏置为 $b^{(1)} \in \mathbb{R}^{1 \times 3}$ ；隐藏层到输出层的权重矩阵为 $W^{(2)} \in \mathbb{R}^{3 \times 8}$ ，偏置为 $b^{(2)} \in \mathbb{R}^{1 \times 8}$ 。

3 激活函数

网络隐藏层和输出层均使用 Sigmoid 激活函数。Sigmoid 函数定义如下：

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

其导数可以表示为：

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{3}$$

在程序实现中，为了防止指数计算时出现数值溢出，代码中使用了 `np.clip(z, -60, 60)` 对输入值进行截断处理。

4 前向传播过程

前向传播过程分为输入层到隐藏层、隐藏层到输出层两个阶段。

首先，输入矩阵 X 经过第一层线性变换，得到隐藏层的预激活值：

$$Z^{(1)} = XW^{(1)} + b^{(1)} \quad (4)$$

然后通过 Sigmoid 激活函数得到隐藏层输出：

$$H = \sigma(Z^{(1)}) \quad (5)$$

其中 $H \in \mathbb{R}^{8 \times 3}$ ，即每个输入样本被编码为 3 维隐藏层表示。

接着，隐藏层输出经过第二层线性变换，得到输出层预激活值：

$$Z^{(2)} = HW^{(2)} + b^{(2)} \quad (6)$$

最后通过 Sigmoid 激活函数得到网络预测输出：

$$O = \sigma(Z^{(2)}) \quad (7)$$

其中 $O \in \mathbb{R}^{8 \times 8}$ ，表示网络解码后的输出矩阵。

5 损失函数

本实验采用均方误差作为损失函数。设训练样本数量为 N ，输出维度为 8，则损失函数定义为：

$$L = \frac{1}{2N} \sum_{n=1}^N \sum_{k=1}^8 (O_{nk} - Y_{nk})^2 \quad (8)$$

程序中的实现为：

$$L = 0.5 \times \text{mean} \left(\sum (O - Y)^2 \right) \quad (9)$$

其中乘以 0.5 是为了在反向传播求导时抵消平方项产生的系数 2，使梯度表达更加简洁。

6 反向传播算法

反向传播的目标是计算损失函数对各层权重和偏置的梯度，然后使用梯度下降法更新参数。

6.1 输出层误差项

输出层误差项为：

$$\Delta^{(2)} = (O - Y) \odot O \odot (1 - O) \quad (10)$$

其中 $\odot$ 表示矩阵对应位置元素相乘。根据链式法则，输出层权重和偏置的梯度为：

$$\frac{\partial L}{\partial W^{(2)}} = \frac{H^T \Delta^{(2)}}{N} \quad (11)$$

$$\frac{\partial L}{\partial b^{(2)}} = \text{mean}(\Delta^{(2)}) \quad (12)$$

6.2 隐藏层误差项

隐藏层误差需要由输出层误差反向传播得到：

$$\Delta^{(1)} = (\Delta^{(2)} (W^{(2)})^T) \odot H \odot (1 - H) \quad (13)$$

因此，输入层到隐藏层权重和偏置的梯度为：

$$\frac{\partial L}{\partial W^{(1)}} = \frac{X^T \Delta^{(1)}}{N} \quad (14)$$

$$\frac{\partial L}{\partial b^{(1)}} = \text{mean}(\Delta^{(1)}) \quad (15)$$

7 梯度下降参数更新

设学习率为 η ，则参数更新公式如下：

$$W^{(2)} \leftarrow W^{(2)} - \eta \frac{\partial L}{\partial W^{(2)}} \quad (16)$$

$$b^{(2)} \leftarrow b^{(2)} - \eta \frac{\partial L}{\partial b^{(2)}} \quad (17)$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \frac{\partial L}{\partial W^{(1)}} \quad (18)$$

$$b^{(1)} \leftarrow b^{(1)} - \eta \frac{\partial L}{\partial b^{(1)}} \quad (19)$$

本实验在每个 epoch 中对完整的 8 个样本进行一次批量梯度下降更新。

8 程序实现说明

本实验代码主要包括四个逻辑部分：参数配置、数学工具函数、自编码器模型类和主程序训练流程。虽然最终提交代码可以保存在一个 `brain_cognitive_autoencoder.py` 文件中，但从结构上可以理解为以下四个部分：

表 1: 程序逻辑模块划分

模块	对应内容	主要功能
<code>config.py</code>	参数配置部分	设置输入维度、隐藏层维度、训练轮数、学习率等
<code>math_utils.py</code>	数学函数部分	实现 Sigmoid 函数和均方误差损失函数
<code>autoencoder.py</code>	模型定义部分	实现前向传播、反向传播、编码、解码和预测
<code>main.py</code>	主程序部分	构造数据、训练模型、保存图像和输出结果

其中，自编码器类 `AutoEncoder838` 是程序的核心。其 `forward` 函数用于完成前向传播，`train_one_epoch` 函数用于完成一次完整的前向传播、损失计算、反向传播和参数更新。

9 实验参数设置

本实验采用的主要超参数如下表所示：

表 2: 实验超参数设置

参数名称	参数取值
输入层神经元数	8
隐藏层神经元数	3
输出层神经元数	8
训练轮数	50000
学习率	10.0
随机种子	42
激活函数	Sigmoid
损失函数	均方误差
优化方法	批量梯度下降

权重初始化采用类似 Xavier 均匀初始化的方法。对于某一层输入维度为 d_{in} 、输出维度为 d_{out} 的权重矩阵，初始化范围为：

$$\left[-\sqrt{\frac{6}{d_{in} + d_{out}}}, \sqrt{\frac{6}{d_{in} + d_{out}}} \right] \quad (20)$$

这种初始化方式有助于避免初始权重过大或过小，从而提升训练稳定性。

10 实验结果与分析

程序运行结束后，会输出最终损失值、原始单位矩阵、隐藏层编码矩阵、解码后的输出矩阵以及经过 0.5 阈值二值化后的矩阵。同时，程序会保存训练过程中的损失函数图像。

在本次实验参数设置下，最终损失值约为：

$$L \approx 7.8142 \times 10^{-5} \quad (21)$$

隐藏层编码矩阵示例如下：

```
[[0.9934 0.0038 0.9310]
 [0.0461 0.0097 0.9917]
 [0.9791 0.9834 0.0044]
 [0.0031 0.0060 0.0675]
 [0.0032 0.9849 0.9752]
 [0.0123 0.9316 0.0063]
 [0.8483 0.0081 0.0040]
 [0.9955 0.9963 0.9962]]
```

解码后的输出矩阵示例如下：

```
[[0.9917 0.0043 0.0000 0.0000 0.0000 0.0000 0.0038 0.0076]
 [0.0057 0.9931 0.0000 0.0048 0.0065 0.0000 0.0000 0.0000]
 [0.0000 0.0000 0.9919 0.0000 0.0000 0.0041 0.0042 0.0073]
 [0.0000 0.0045 0.0000 0.9919 0.0000 0.0056 0.0062 0.0000]
 [0.0000 0.0040 0.0000 0.0000 0.9915 0.0038 0.0000 0.0076]
 [0.0000 0.0000 0.0059 0.0042 0.0066 0.9927 0.0000 0.0000]
 [0.0066 0.0000 0.0061 0.0041 0.0000 0.0000 0.9922 0.0000]
 [0.0040 0.0000 0.0040 0.0000 0.0036 0.0000 0.0000 0.9888]]
```

将输出矩阵按照阈值 0.5 进行二值化后，得到：

```
[[1 0 0 0 0 0 0 0]
 [0 1 0 0 0 0 0 0]
 [0 0 1 0 0 0 0 0]
 [0 0 0 1 0 0 0 0]
 [0 0 0 0 1 0 0 0]
 [0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 1]]
```

由结果可以看出，解码后的矩阵在主对角线位置接近 1，在非主对角线位置接近 0。经过阈值化处理后，输出矩阵能够恢复为 8×8 单位矩阵。这说明该 8-3-8 自编码器已经基本学会了对单位矩阵的编码与解码映射。

11 损失函数曲线

训练过程中，程序将每一轮 epoch 的损失值保存下来，并绘制为损失函数曲线。随着训练轮数增加，损失函数整体呈下降趋势，说明网络参数通过反向传播和梯度下降不断被优化。

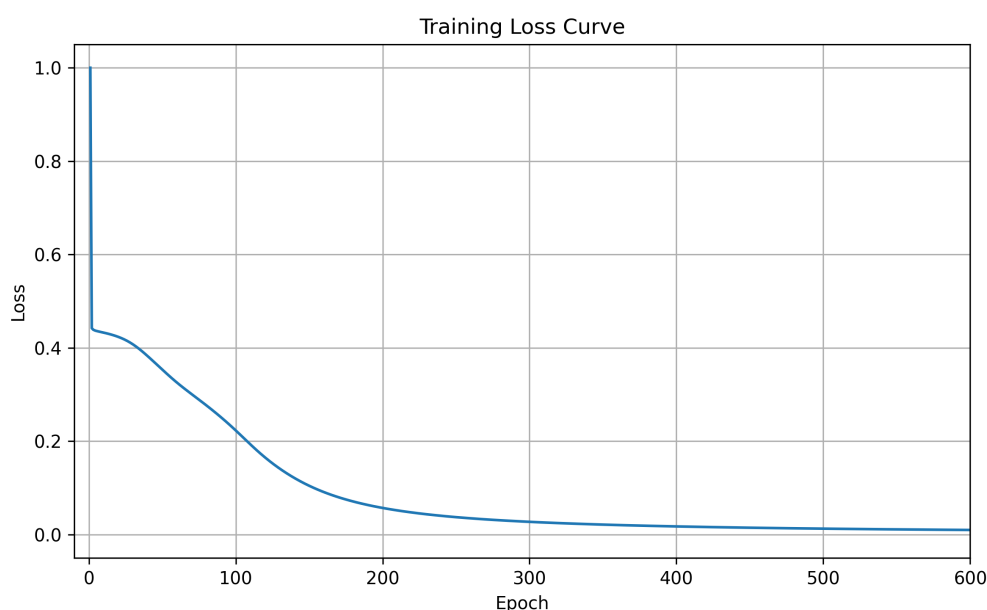


图 1: 训练过程中的损失函数曲线

从图1可以观察训练过程中损失函数的变化情况。若曲线逐渐下降并趋于稳定，说明模型的训练过程是有效的。

12 结论

本文使用 Python 手动实现了一个 8-3-8 结构的自编码器神经网络。实验中没有使用 TensorFlow、PyTorch 等深度学习框架，而是通过 NumPy 完成矩阵运算、前向传播、反向传播和梯度下降更新。模型以 8×8 单位矩阵作为训练数据，通过 3 个隐藏层神经元完成压缩编码，并最终在输出层恢复原始输入。

实验结果表明，经过 50000 轮训练后，网络损失降低到较小水平，解码输出矩阵与原始单位矩阵高度接近。经过阈值化处理后，输出结果可以恢复为标准单位矩阵，说明该神经网络成功实现了对 8×8 单位矩阵的编解码功能。